

20 Exercícios Resolvidos de Python

Listas • Matrizes (listas dentro de listas) • Dicionários • Compreensão de listas e dicionários

Material de estudo com exemplos progressivos. As soluções são completas e comentadas por meio de observações após cada exercício.

Como estudar

Uma boa forma de aproveitar os exercícios é tentar resolver o enunciado antes de consultar a solução. Depois, compare sua implementação com o gabarito e procure entender não apenas o resultado, mas a estrutura usada.

Conteúdos trabalhados: criação e manipulação de listas, índices, slicing, `append()`, `in`, `not in`, `sum()`, `len()`, `max()`, `min()`, listas aninhadas, percursos com dois `for`, diagonais, transposição, dicionários, `items()`, listas de dicionários, compreensão de listas e compreensão de dicionários.

1. Soma e média de uma lista

Crie uma lista com 8 notas e calcule a soma e a média das notas.

Solução:

```
notas = [7.5, 8.0, 6.5, 9.0, 5.5, 7.0, 8.5, 6.0]
```

```
soma = sum(notas)
media = soma / len(notas)
```

```
print(f"Soma: {soma}")
print(f"Média: {media:.2f}")
```

Observação: Revisão: `sum()` soma os elementos da lista e `len()` informa a quantidade de elementos.

2. Separando números pares e ímpares

Dada uma lista de números inteiros, crie duas novas listas: uma contendo os pares e outra contendo os ímpares.

Solução:

```
numeros = [3, 8, 12, 7, 15, 20, 4, 9, 11, 6]
```

```
pares = []
impares = []
```

```
for numero in numeros:
    if numero % 2 == 0:
        pares.append(numero)
    else:
        impares.append(numero)
```

```
print("Pares:", pares)
print("Ímpares:", impares)
```

Observação: Observe que `append()` acrescenta um elemento ao final da lista.

3. Maior, menor e posição dos valores

Encontre o maior e o menor valor de uma lista e informe suas posições (índices).

Solução:

```
numeros = [18, 7, 25, 3, 42, 11, 9]
```

```
maior = max(numeros)
menor = min(numeros)
```

```
print("Maior:", maior)
print("Índice do maior:", numeros.index(maior))
print("Menor:", menor)
print("Índice do menor:", numeros.index(menor))
```

Observação: Os índices começam em 0. `index()` retorna a primeira posição em que o valor aparece.

4. Removendo valores repetidos

Receba uma lista que pode possuir números repetidos e gere outra lista contendo cada número apenas uma vez, mantendo a ordem original.

Solução:

```
numeros = [4, 2, 4, 7, 2, 9, 7, 1, 4]
```

```
unicos = []
```

```
for numero in numeros:
    if numero not in unicos:
```

```
unicos.append(numero)
```

```
print(unicos)
```

Observação: Esta solução é didática porque mostra claramente o uso de in e not in.

5. Invertendo uma lista

Crie uma nova lista com os elementos de outra lista em ordem inversa.

Solução:

```
nomes = ["Ana", "Bruno", "Carlos", "Daniel", "Eduardo"]
```

```
invertida = nomes[::-1]
```

```
print("Original:", nomes)
print("Invertida:", invertida)
```

Observação: A sintaxe lista[::-1] usa fatiamento (slicing) para percorrer a lista de trás para frente.

6. Compreensão de listas: quadrados

Usando compreensão de listas, crie uma lista contendo o quadrado dos números de 1 a 10.

Solução:

```
quadrados = [numero ** 2 for numero in range(1, 11)]
```

```
print(quadrados)
```

Observação: Forma geral: [expressão for variável in sequência]. A compreensão de listas é uma maneira compacta de construir listas.

7. Compreensão de listas com condição

A partir de uma lista de idades, crie uma nova lista contendo somente as idades maiores ou iguais a 18.

Solução:

```
idades = [12, 18, 21, 15, 32, 17, 19, 14, 25]
```

```
maiores = [idade for idade in idades if idade >= 18]
```

```
print(maiores)
```

Observação: A condição fica depois do for: [expressão for variável in sequência if condição].

8. Matriz 3 × 3: exibindo os elementos

Crie uma matriz 3 × 3 e percorra todos os seus elementos usando dois for.

Solução:

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

```
for linha in matriz:
    for elemento in linha:
        print(elemento, end=" ")
    print()
```

Observação: Uma matriz é representada por uma lista cujos elementos também são listas. O primeiro for percorre as linhas; o segundo percorre os elementos de cada linha.

9. Soma de todos os elementos de uma matriz

Calcule a soma de todos os números de uma matriz 3×3 .

Solução:

```
matriz = [  
    [5, 2, 8],  
    [1, 7, 3],  
    [4, 6, 9]  
]  
  
soma = 0  
  
for linha in matriz:  
    for elemento in linha:  
        soma += elemento  
  
print("Soma:", soma)
```

Observação: Este exercício reforça o padrão de dois laços para percorrer listas dentro de listas.

10. Diagonal principal

Dada uma matriz quadrada 4×4 , mostre os elementos da diagonal principal.

Solução:

```
matriz = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12],  
    [13, 14, 15, 16]  
]  
  
for i in range(len(matriz)):  
    print(matriz[i][i])
```

Observação: Na diagonal principal, linha e coluna possuem o mesmo índice: [0][0], [1][1], [2][2] etc.

11. Soma de cada linha de uma matriz

Calcule a soma individual de cada linha de uma matriz.

Solução:

```
matriz = [  
    [2, 4, 6],  
    [1, 3, 5],  
    [10, 20, 30]  
]  
  
for linha in matriz:  
    soma = sum(linha)  
    print("Soma da linha:", soma)
```

Observação: Como cada linha já é uma lista, podemos aplicar `sum()` diretamente sobre ela.

12. Encontrando o maior valor de uma matriz

Encontre o maior elemento de uma matriz sem usar `max()` diretamente sobre a matriz inteira.

Solução:

```
matriz = [  
    [12, 5, 8],  
    [21, 4, 17],
```

```
[9, 30, 6]
]
```

```
maior = matriz[0][0]
```

```
for linha in matriz:
    for elemento in linha:
        if elemento > maior:
            maior = elemento
```

```
print("Maior elemento:", maior)
```

Observação: A variável maior começa com o primeiro elemento e vai sendo atualizada quando encontramos um valor superior.

13. Transpondo uma matriz com compreensão de listas

Dada uma matriz 2 × 3, crie sua transposta, que terá dimensão 3 × 2, usando compreensão de listas.

Solução:

```
matriz = [
    [1, 2, 3],
    [4, 5, 6]
]

transposta = [
    [matriz[linha][coluna] for linha in range(len(matriz))]
    for coluna in range(len(matriz[0]))
]

print(transposta)
```

Observação: Na transposição, as colunas da matriz original tornam-se linhas da nova matriz. Este exercício combina matrizes e compreensão de listas.

14. Cadastro simples com dicionário

Crie um dicionário contendo nome, idade e cidade de uma pessoa e exiba os dados.

Solução:

```
pessoa = {
    "nome": "Carlos",
    "idade": 28,
    "cidade": "São Carlos"
}

print("Nome:", pessoa["nome"])
print("Idade:", pessoa["idade"])
print("Cidade:", pessoa["cidade"])
```

Observação: Dicionários armazenam pares chave: valor. As chaves permitem acessar diretamente os dados.

15. Contagem de palavras

Dada uma lista de palavras, crie um dicionário que informe quantas vezes cada palavra aparece.

Solução:

```
palavras = [
    "python", "java", "python", "c",
    "python", "java", "c", "python"
]
```

```
contagem = {}
```

```
for palavra in palavras:
    if palavra in contagem:
        contagem[palavra] += 1
    else:
        contagem[palavra] = 1
```

```
print(contagem)
```

Observação: Este é um padrão muito importante: verificar se uma chave já existe antes de incrementar seu valor.

16. Compreensão de dicionários: quadrados

Crie um dicionário em que as chaves sejam os números de 1 a 10 e os valores sejam seus respectivos quadrados.

Solução:

```
quadrados = {
    numero: numero ** 2
    for numero in range(1, 11)
}
```

```
print(quadrados)
```

Observação: A compreensão de dicionários segue a ideia {chave: valor for variável in sequência}.

17. Compreensão de dicionários com condição

A partir de um dicionário de produtos e preços, crie outro dicionário contendo somente os produtos que custam menos de R\$ 50.

Solução:

```
produtos = {
    "caneta": 3.50,
    "caderno": 28.90,
    "mochila": 85.00,
    "borracha": 4.50,
    "estojo": 35.00
}

baratos = {
    produto: preco
    for produto, preco in produtos.items()
    if preco < 50
}
```

```
print(baratos)
```

Observação: items() permite percorrer simultaneamente a chave e o valor. A condição filtra os elementos.

18. Dicionário de alunos e médias

Crie um dicionário contendo o nome de cada aluno e sua média. Depois, crie outro dicionário apenas com os alunos aprovados (média ≥ 7).

Solução:

```
medias = {
    "Ana": 8.5,
    "Bruno": 6.2,
    "Carlos": 7.0,
    "Daniela": 9.1,
    "Eduardo": 5.8
}

aprovados = {
```

```

nome: media
for nome, media in medias.items()
if media >= 7
}

```

```

print("Aprovados:", aprovados)

```

Observação: Este exercício combina dicionários, items() e compreensão de dicionários com condição.

19. Lista de dicionários: cadastro de alunos

Crie uma lista contendo dicionários, em que cada dicionário representa um aluno. Depois, mostre apenas os alunos com média ≥ 7 .

Solução:

```

alunos = [
    {"nome": "Ana", "idade": 20, "media": 8.5},
    {"nome": "Bruno", "idade": 22, "media": 6.0},
    {"nome": "Carlos", "idade": 19, "media": 7.2},
    {"nome": "Daniela", "idade": 21, "media": 9.0}
]

```

```

aprovados = [
    aluno
    for aluno in alunos
    if aluno["media"] >= 7
]

```

```

for aluno in aprovados:
    print(aluno["nome"], aluno["media"])

```

Observação: Aqui temos uma estrutura muito comum em programas reais: uma lista de registros, sendo cada registro um dicionário.

20. Exercício integrado: matriz de produtos e dicionário de estatísticas

Considere uma lista de dicionários representando produtos. Para cada produto há uma matriz de vendas, em que cada linha representa uma semana e cada coluna representa um dia.

Calcule, para cada produto, o total vendido e crie um dicionário de resultados. Depois, use compreensão de dicionários para selecionar os produtos com total superior a 20.

Solução:

```

produtos = [
    {
        "nome": "Caderno",
        "vendas": [
            [2, 3, 1, 4, 2],
            [3, 2, 4, 1, 3]
        ]
    },
    {
        "nome": "Caneta",
        "vendas": [
            [5, 4, 3, 2, 4],
            [3, 5, 4, 2, 3]
        ]
    },
    {
        "nome": "Mochila",
        "vendas": [

```

```

        [1, 2, 2, 1, 3],
        [2, 1, 2, 2, 1]
    ]
}
]

totais = {}

for produto in produtos:
    total = 0

    for semana in produto["vendas"]:
        total += sum(semana)

    totais[produto["nome"]] = total

print("Totais:", totais)

acima_de_20 = {
    nome: total
    for nome, total in totais.items()
    if total > 20
}

print("Produtos com mais de 20 unidades:", acima_de_20)

```

Observação: Este exercício reúne os três temas: lista, matriz (lista dentro de lista), dicionário e compreensão de dicionários. É um bom exercício final para revisar a relação entre as estruturas.